


# LAB TASK # 11

NAME : MASHAAL ISHTIAQ

ROLL NO : 22P-9326

# QUESTION#1

```
#include <iostream>
using namespace std;

class Node
{

private:
    int val;
    Node *left;
    Node *right;
    int height;

public:
    Node(int val)
    { // parameterized constructor
        this->val = val;
        left = right = NULL;
        height = 1;
    }

    friend class AVL;
    friend int main();
};

class AVL
{
private:
    // public:
    Node *root;

public:
    AVL()
    {
        root = NULL;
    }

    Node *insert(Node *node, int val)
    {
        if (node == NULL)
        {
            Node *temp = new Node(val);
            if (root == NULL)
            {
                root = temp;
            }

            return temp;
        }

        if (val < node->val)
        {
            node->left = insert(node->left, val);
        }
        else if (val > node->val)
        {
            node->right = insert(node->right, val);
        }
```

```
        node->height = updateHeight(node);
        return node;
    }

    int getHeight(Node *node)
    {
        if (node == NULL)
            return 0;

        return node->height;
    }

    int updateHeight(Node *node)
    {
        if (node == NULL)
            return 0;

        int rightH = getHeight(node->right);
        int leftH = getHeight(node->left);

        return 1 + max(leftH, rightH); // return 1+[(left > right) ? left :
right ]
    }

    bool findAVL(Node *node)
    {
        if (node == NULL)
            return true;

        int rightH = getHeight(node->right);
        int leftH = getHeight(node->left);

        if ((leftH - rightH) <= 1 && (leftH - rightH) == -1 && findAVL(node-
>left) && findAVL(node->right))
        {
            return true;
        }

        return false;
    }
    friend int main();
};

int main()
{
    AVL avl;
    // hard code

    avl.root = avl.insert(avl.root, 50);
    avl.insert(avl.root, 30);
    avl.insert(avl.root, 70);
    avl.insert(avl.root, 20);
    avl.insert(avl.root, 40);
    avl.insert(avl.root, 60);
    avl.insert(avl.root, 80);

    if (avl.findAVL(avl.root))
    {
        cout << "AVL" << endl;
    }
    else
    {
        cout << "Not AVL" << endl;
```

```
    }
}
```

```
Q1 && "/home/mashaal/Desktop/semster_4/data_structu
AVL

[Done] exited with code=0 in 0.411 seconds
```

# QUESTION#2

```
#include <iostream>
using namespace std;

class Node
{

private:
int val;
Node *left;
Node *right;
int height;

public:
Node(int val)
{ // parameterized constructor
this->val = val;
left = right = NULL;
height = 1;
}

friend class AVL;
friend int main();
};

class AVL
{
private:
// public:
Node *root;

public:
```

```
AVL()
{
root = NULL;
}

Node *insert(Node *node, int val)
{
if (node == NULL)
{
Node *temp = new Node(val);
if (root == NULL)
{
root = temp;
}

return temp;
}

if (val < node->val)
{
node->left = insert(node->left, val);
}
else if (val > node->val)
{
node->right = insert(node->right, val);
}

node->height = updateHeight(node);
int balance = getBalance(node);

if (balance > 1) // -2 , -1 , 0 , 1 ,2 (its depending on number line therefore leftRotate less than 1 and
the greater than 0 rightRotate )
{
if (val < node->left->val) // LL case
{
return rightRotate(node);
}

else // LR
{
node->left = leftRotate(node->left);
return rightRotate(node);
}
}

else if (balance < -1)
{

if (val > node->right->val) // RR
{
return leftRotate(node);
}
```

```
else // RL
{
node->right = rightRotate(node->right);
return leftRotate(node);
}
}
return node; // if balance is -1 , 0 ,1 -> no rotation
}

// ___ height____

int getHeight(Node *node)
{
if (node == NULL)
return 0;

return node->height;
}

int updateHeight(Node *node)
{
if (node == NULL)
return 0;

int rightH = getHeight(node->right);
int leftH = getHeight(node->left);

return 1 + max(leftH, rightH); // return 1+[(left > right) ? left : right ]
}

//_____balance_____

int getBalance(Node *node)
{
if (node == NULL)
{
return 0;
}
return getHeight(node->left) - getHeight(node->right);
}

//____rotations___

Node *rightRotate(Node *R) // R->root of the // reference is given to make changes in the original as welltree being rotated
{

Node *temp = R->left;
R->left = temp->right;
temp->right = R;
R->height = updateHeight(R);
```

```
temp->height = updateHeight(temp);
return temp;
}

Node *leftRotate(Node *&R)
{
Node *temp = R->right;
R->right = temp->left;
temp->left = R;
R->height = updateHeight(R);
temp->height = updateHeight(temp);
return temp;
}

void display(Node *node) // displaying using inorder traversal
{
if (node != NULL)
{
display(node->left);
cout << node->val << " ";
display(node->right);
}
}

// bool findAVL(Node *node)
// {
// if (node == NULL)
// return true;

// int rightH = getHeight(node->right);
// int leftH = getHeight(node->left);

// if ((leftH - rightH) <= 1 && (leftH - rightH) == -1 && findAVL(node->left) && findAVL(node->right))
// {
// return true;
// }

// return false;
// }
friend int main();
};

int main()
{
AVL avl;
// hard code
avl.root = NULL;

avl.insert(avl.root, 50);
avl.insert(avl.root, 30);
avl.insert(avl.root, 70);
```

```
avl.insert(avl.root, 20);
avl.insert(avl.root, 40);
avl.insert(avl.root, 60);
avl.insert(avl.root, 80);

// displaying the nodes

cout << "____AVL tree___\n";
avl.display(avl.root);
cout << endl;

return 0;
}
```

